

MASTER README — every folder, every subfolder

The two map files, and the difference between them

>

- **MASTER README** (this file) describes **every folder and every subfolder**, branch by branch, all the way down. Read it to find a specific file.
- **LOCAL README** ([LOCAL README.md](#)) describes ****only the folders at the top level****. One line each, no going inside them. Read it to find out which door to open.

>

A third file, [PROJECT OVERVIEW.md](#), is not a map: it describes the machine itself — what it is, the firmware, the bills of material, the DIC channel, and how to get the software running.

****The final report is at**

[Documentation/Final Report/UTM DIC Report.pdf](#)

— 99 pages, with the editable [UTM_DIC_Report.docx](#) beside it.

The project: a 1-D digital image correlation strain channel built onto the Department of Product Development, Production and Design's desktop universal testing machine at Jönköping University's School of Engineering, validated against a certified video extensometer and applied to PLA, PETG and TPU. February – September 2026.

The folder tree

Drawn from the directory tree itself by [Documentation/Scripts/master_readme_pdf.py](#), so it cannot drift from the repository. The number after each folder is how many files sit directly in it.

The repository, two levels deep (1 of 2)

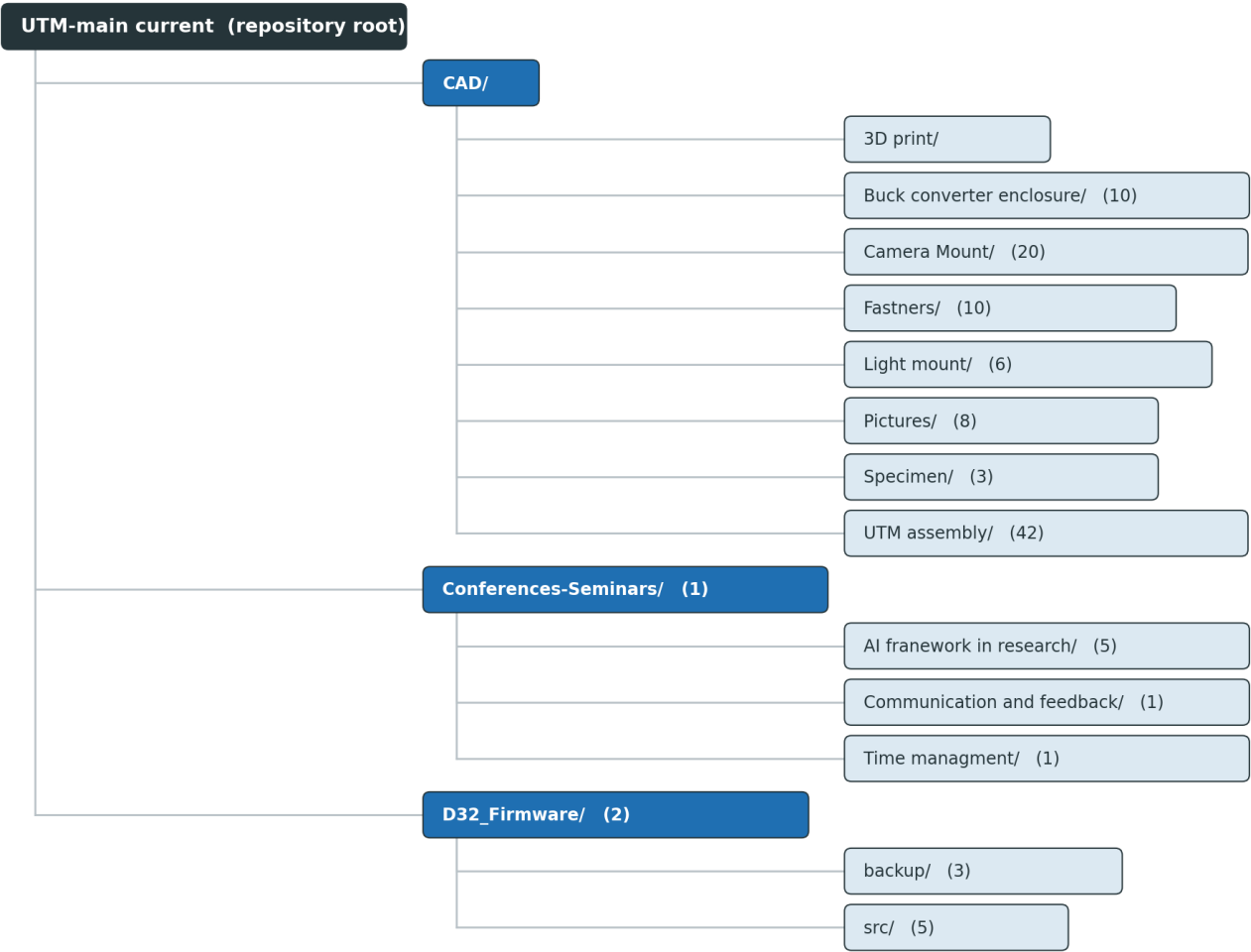


Figure 1 — The repository, two levels deep (1 of 2)

The repository, two levels deep (2 of 2)

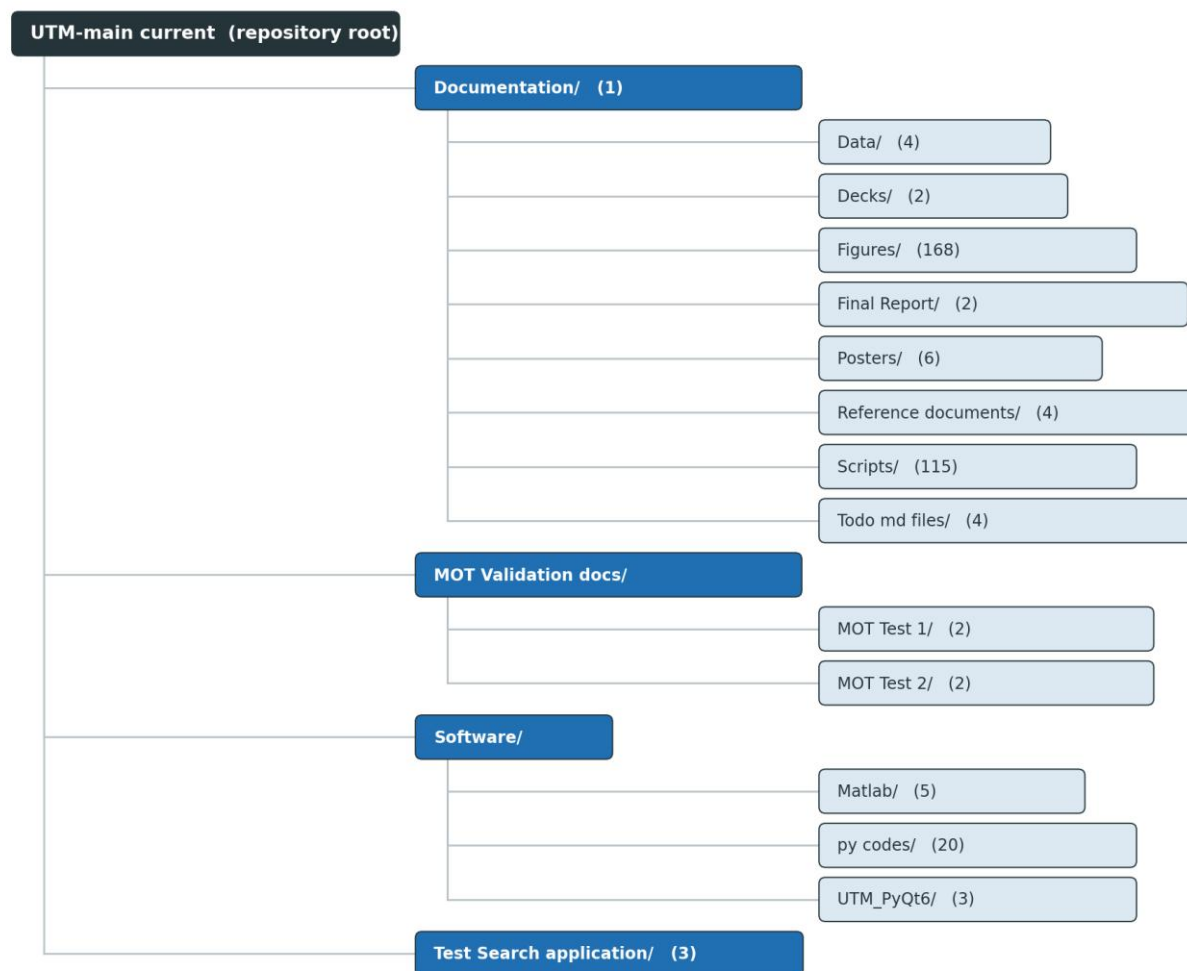


Figure 2 — The repository, two levels deep (2 of 2)

Documentation/, expanded (1 of 2)

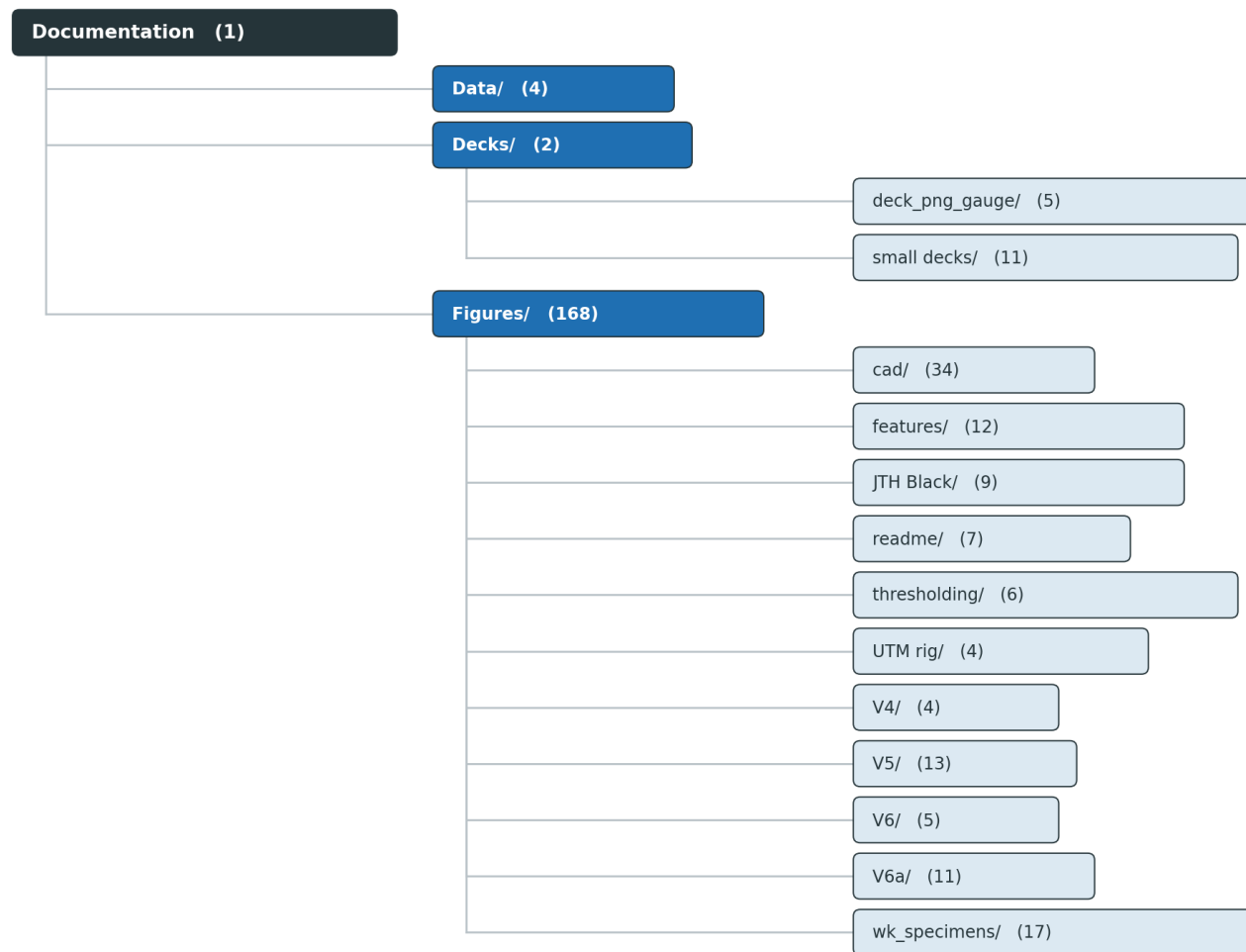


Figure 3 — Documentation/, expanded (1 of 2)

Documentation/, expanded (2 of 2)

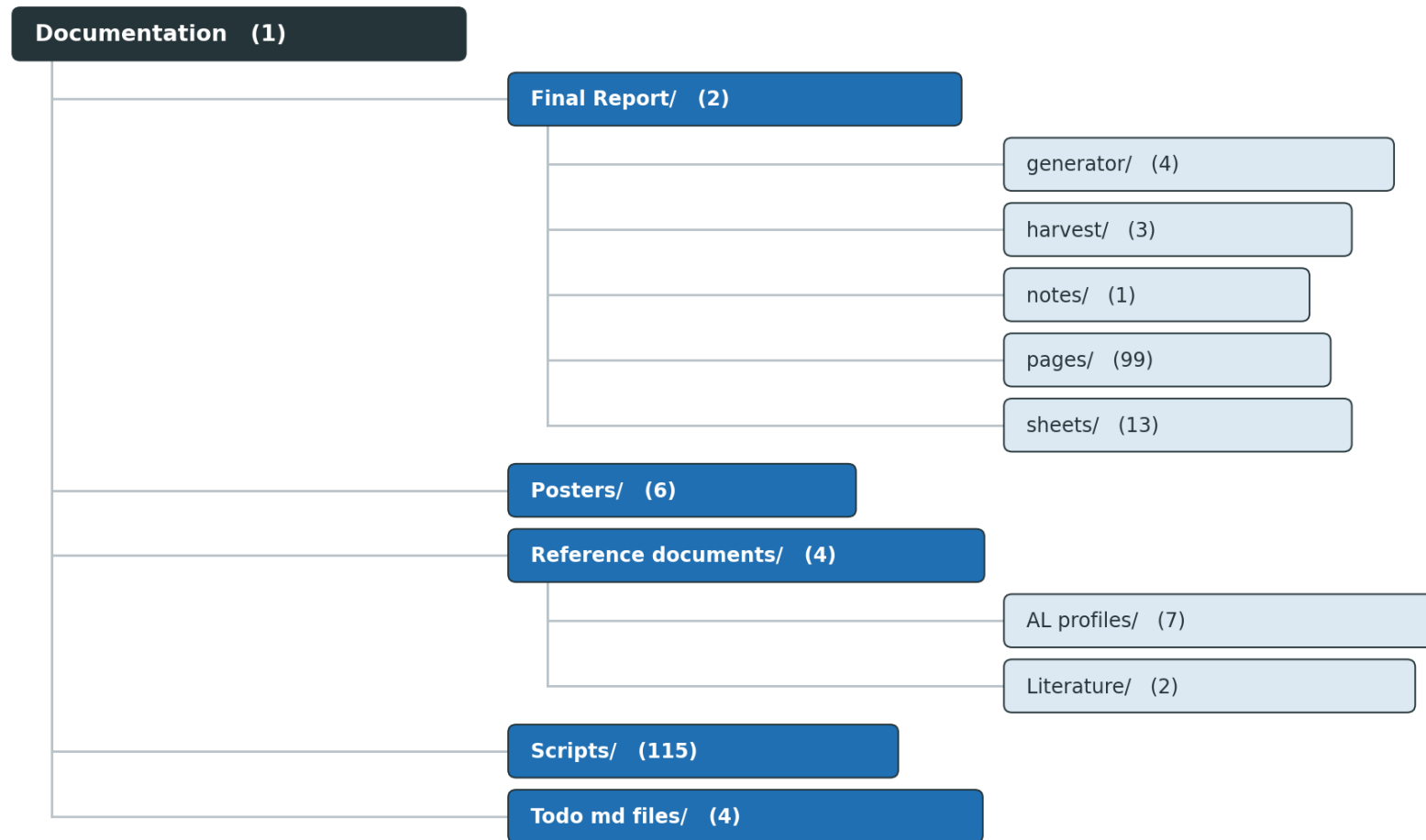


Figure 4 — Documentation/, expanded (2 of 2)

Software/, expanded

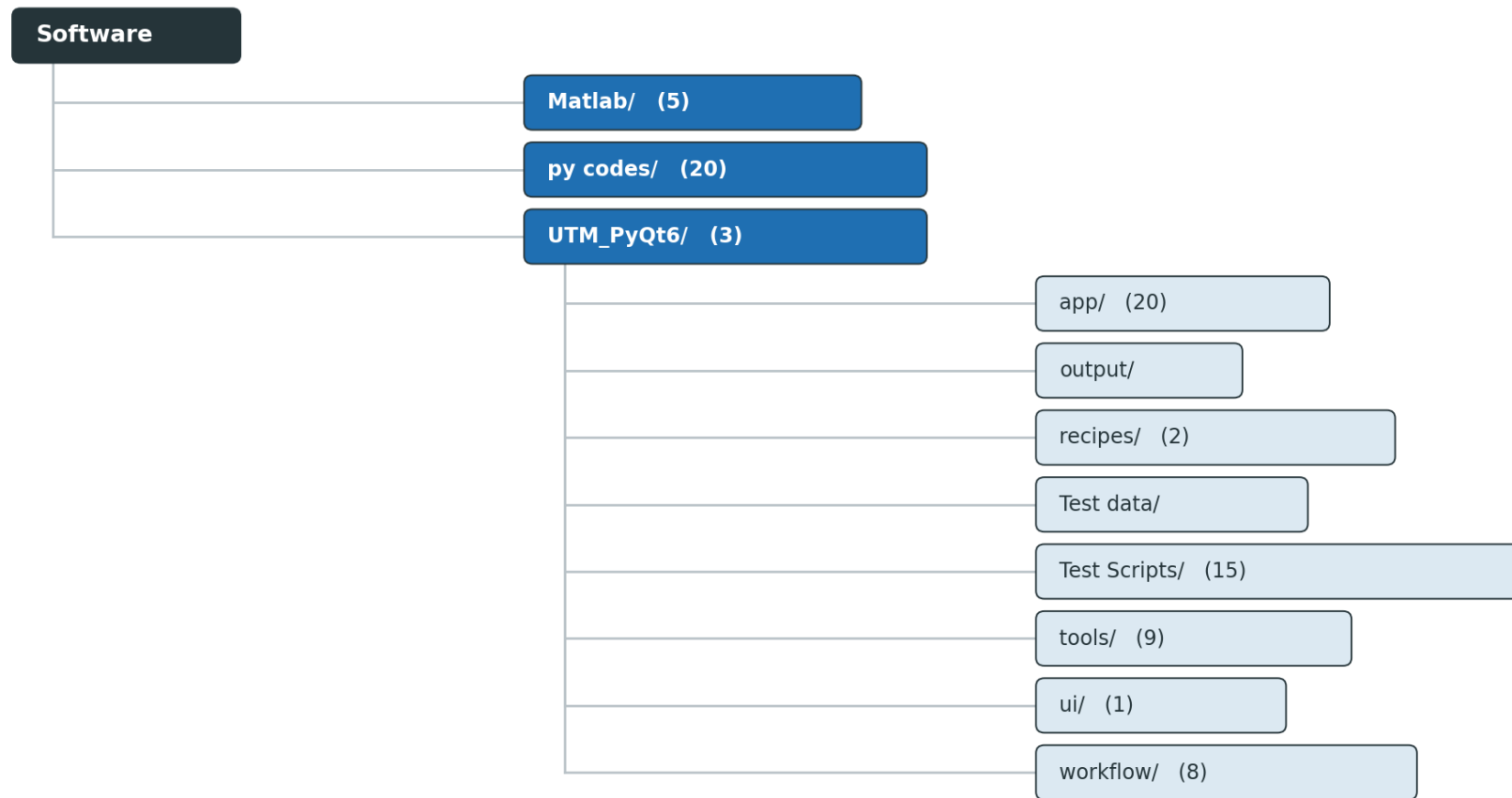


Figure 5 — Software/, expanded

CAD/ — everything designed for the rig

subfolder	what is in it
UTM assembly/	The rig as a whole: SolidWorks parts and assemblies, STEP and STL exports. The largest folder here — the frame, grips, base and mounts.
Camera Mount/	The camera carriage and its rail: parts, assemblies and DXF profiles. The rib and rail-profile studies in the report come from these.
Buck converter enclosure/	The electronics box that holds the buck converter, with STEP exports.
Light mount/	The printed uprights and diffuser mount for the LED strip.
Specimen/	The tensile specimen models and the marker-template assemblies.
Fasteners/	Bought-in fasteners and inserts, kept as models so assemblies resolve.
Pictures/	Photographs of the printed and assembled parts.
3D print/	What was actually sent to the printer — Stl/ , Gcode/ , plus a Laptop stand/ side print.

Conferences-Seminars/ — seminars and department days

subfolder	what is in it
AI framework in research/	Slides and notes from the AI-in-research seminar.
Communication and feedback/	Seminar material on communication and feedback.
Time management/	Seminar material on time management.

A department-days PDF sits at the folder root. Nothing here feeds the rig or the report.

D32_Firmware/ — the ESP32 firmware

A PlatformIO project. [platformio.ini](#) at the root, and:

subfolder	what is in it
src/	The firmware itself: C++ sources and headers — HX711 load-cell polling at 10 Hz, two TMC2160 drivers on one control signal, AS5600 encoders on the motor shafts.
backup/	The previous Arduino-sketch version, kept for reference.
.vscode/	Editor and build configuration for the firmware project.

Documentation/ — everything written or drawn about the project

[README.md](#) at the root of this folder is its own local map.

subfolder	what is in it
Final Report/	The report. See the breakdown below.
Decks/	The presentation decks. See the breakdown below.
Figures/	Every figure any deck, poster or the report reads or writes. See the breakdown below.

Scripts/	110 Python files: the deck builders, the poster builder, the report-figure and deck-surgery scripts, and the data and plot modules they import. Run them from the repository root — every path inside is written relative to it.
Data/	The computed caches the builders read: estimator_measurements.json , mot_postproc_compare.json , pxmm_reasoning.json , mot_matched_points.csv . Written by mot_postproc_compare.py , estimator_data.py and pxmm_data.py .
Posters/	Smart_UTM_poster_A0 , Smart_UTM_poster_A4 and Smart_UTM_progress_A4 , each as .pptx and .pdf .
Reference documents/	The literature read during the project: the iDICs *Good Practices Guide* Edition 2 (GPG_Ed2_251104.pdf), the predecessor thesis (FULLTEXT01.pdf), the individual design report, a DIC study guide, plus Literature/ (papers) and AL_profiles/ (supplier photographs of the aluminium profile).
Todo md files/	Project-level documents: TODO.md , Starting_TODO.md , PROJECT_REQUIREMENTS.md , VERSIONING.md .

Documentation/Final Report/

The two deliverables sit alone at the root — [UTM_DIC_Report.docx](#) and

[UTM_DIC_Report.pdf](#) (99 pages).

subfolder	what is in it
generator/	The generator: build_report.py (two-pass compose, then Word COM for the PDF), rp.py (the document plumbing — headings, figures, tables, cross-references, citations), refs.py (the bibliography, 57 entries), harvest_refs.py (pulls the links out of the decks), and ch/ — one module per chapter plus the front matter and appendices. Rebuild with <code>cd "Documentation/Final Report/generator" && python build_report.py</code> .
pages/	One PNG per page of the built PDF, for reviewing layout.
sheets/	Contact sheets — the same pages tiled, for a whole-document glance.
harvest/	Artifacts, not inputs: refs_harvest.json , refs_harvest.md , and registry_snapshot.json (the test registry as it stood when the report was built; the appendix's specimen register reads it).
notes/	REPORT_PLAN.md — the plan the report was written against.

Documentation/Decks/

[Weekly progress updated.pptx](#) is the live progress diary (397 slides) and

[PPD_Final_Presentation.pptx](#) is the final departmental presentation.

subfolder	what is in it
small decks/	The single-topic decks: Learnings.pptx (the pre-study digest of the DIC guide), the V4/V5/V5abc/V6a test-series decks, V2_capture_validation.pptx , UTM DIC Implementation.pptx , E_modulus_explained , and the MOT and S25/S26 reference packs.
deck_png_gauge/	Slide renders kept for measuring text fit before a deck edit.

Documentation/Figures/

158 PNGs and 9 JPGs at the root — everything the report and the current decks read, written

by the scripts in [Documentation/Scripts/](#). [README.md](#) here explains the naming.

subfolder	what is in it
cad/	Design figures extracted from the deck for the report's CAD chapter.
V4/, V5/, V6/, V6a/	The test-series plots, by series: staircase and linearity (V4), tensile-to-failure and the V5a/b/c set (V5), the V6 quintet (V6), and the 100 % infill series (V6a).
features/	The feat_* proof figures for the twenty smart features, plus the UI screenshots.
UTM rig/	Hardware and UI: the GUI, the rig schematic, the dimensioned drawing, the enclosure CAD render. Embedded in PROJECT_OVERVIEW.md and PROJECT_REQUIREMENTS.md .
wk_specimens/	Specimen photographs used across the weekly deck.
thresholding/	The binary_*.png threshold sweep — the same frame at six threshold values.
JTH Black/	Jönköping University logos, in EPS, SVG, PNG and JPG.

MOT Validation docs/ — the external validation

The two sessions on the certified Zwick/Roell Z100 with the XT-205 video extensometer, on

which the whole validation chapter rests.

subfolder	what is in it
MOT Test 1/	The first session (80 mm gauge). README.txt , registry.json , plus Videoextesometer MOT/ (the laboratory's own record and footage), DIC post processing data/ (what the rig's post-processor made of it) and PPD UTM DIC rig reference docs/ (the matching rig runs).
MOT Test 2/	The second session (45 mm gauge) — the one Analysis 1 and Analysis 2 in the report use. Same structure.

Software/ — all the code

subfolder	what is in it
UTM_PyQt6/	The application that runs the rig, its tests, its raw test data and its written record. Broken down below.
py_codes/	19 standalone analysis scripts, one per test series or question. Broken down below.
Matlab/	Where the project started — the original App Designer application. Broken down below.

Software/UTM_PyQt6/ — the application

[README.md](#), [registry.json](#) (the test registry — every run the rig has recorded) and

[requirements.txt](#) sit at the root. Nothing else does: all Python lives in [app/](#).

app/ — the 20 application modules

[main.py](#) is the entry point: `python "Software/UTM_PyQt6/app/main.py"` from the repository

root. The modules import each other by bare name, so they must stay together.

module	what it does
main.py	The main window and the whole live path: the state machine (idle → preload → running → fracture → release), the control loop, the live plots and HUD, CSV export, the guards, and the UI for all twenty smart features. By far the largest module.
camera_manager.py	The Basler camera: pylon integration, ROI, exposure and gain, the frame sink, and the latest-image strategy that keeps the loop from falling behind.
utm_dic.py	The DIC core — thresholding, marker segmentation, centroid, and the pixel-to-strain rule. Small and deliberately stable.
utm_analysis.py	The canonical analyser: fit window, yield, UTS, modulus, fracture detection, toughness. The app and every script import this one; nothing re-implements it.
serial_manager.py	The serial link to the ESP32 — force and position at the control period, and the command handler.
control_policies.py	The six closed-loop protocols (staircase, cyclic, creep, relaxation, progressive cyclic, staircase-to-fracture) as policy objects.
utm_postproc.py	The offline engine: re-measures a recorded video with the same DIC code the live path uses.
utm_postdlg.py	The post-processing dialog around that engine — the largest UI module after main.py .
utm_capture.py / utm_capdlg.py	Video recording alongside a test, and its settings dialog.
utm_camdlg.py	The camera settings dialog.
utm_autocal.py	Automatic pixels-per-millimetre calibration from a known marker separation.
utm_blobpick.py	Manual marker selection — the feature that closed the gap when automatic detection failed.
utm_noisedlg.py	The noise-floor measurement dialog.
utm_wizard.py	The guided setup wizard.
utm_recipes.py	Material recipes: load, save and apply a full acquisition setup.
utm_registry.py	The test registry — scans runs, adds rows, resolves paths. Also a CLI.
utm_report.py	The per-test PDF report, from the CSV. Also a CLI.
widgets.py / theme.py	The custom widgets (gauges, toggles) and the visual theme.

Test data/ — the raw runs

Every CSV, and therefore every number in the report, traces back here.

subfolder	what is in it
Fracture tests/	29 runs — the tensile-to-failure series across PLA, PETG and TPU (Specimen_S10... upward), each with its CSV, capture and index.
Smart Features - Advanced Test Modes data/	12 runs — one per closed-loop protocol: creep, relaxation, the cyclic modes, the staircases, progressive cyclic to fracture.
Trial for lateral strain/	2 runs, S38 and S39 — the Poisson's-ratio trial that the report's future work builds on.

Test Scripts/ — 15 test scripts

Five run offline with no hardware: [test_threshold.py](#), [test_postproc_guards.py](#), [test_scale_sidecar.py](#), [test_multipair_postproc.py](#), [test_plot_cursor.py](#). The rest need the camera or the rig: [test_camera.py](#) through [test_camera4.py](#), [test_blob.py](#), [test_manual_blobs.py](#), [test_reacquire.py](#), [test_phase_8_6.py](#) with its [validate_phase_8_6.py](#).

tools/ — developer tools beside the app

tool	what it does
control_sim.py	The simulation plant the closed-loop controllers were tuned on, without touching a specimen.
dic_replay.py	Replays a recorded video through the DIC code for debugging.
camera_setup.py	Interactive camera setup and exposure checking.
check_blobs.py / roi_tool.py	Marker-detection and ROI inspection on recorded frames.
analyze_hardware_tests.py	Batch analysis of the hardware test runs.
annotate_utm.py	Annotates rig photographs for the documentation.
generate_mode_help.py	Generates the per-protocol help images that ui/help/ holds.
build_exe.py	Packages the application as a standalone executable.

workflow/ — the written record

[ROADMAP.md](#) (**start here** — the living status of every feature: done, planned, blocked, and what needs rig time), [TESTING_TODO.md](#) (the rig-test checklist the validations were run against), [TEST_FAILURES.md](#) (what failed and why, S15's stall included), [PYTHON_WORKFLOW.md](#) and its PDF (one page per module), [RECALIBRATE_ROI.md](#) (the procedure after the camera moves), [COMMANDS.txt](#) (the serial command set) and [utm_code_summary.txt](#) (a per-file summary by functional area).

ui/, recipes/, output/

subfolder	what is in it
ui/	utm_mainwindow.ui , the Qt Designer layout main.py loads, and help/ — six PNGs, one per closed-loop protocol, shown in the app's own help.
recipes/	Material recipes as plain JSON: Default.json and TPU.json . Readable, diffable, easy to hand-edit.
output/	What the app writes at run time: captures/ (recorded video), diagnostics/ (threshold and blob dumps when something looks wrong), full_frame_output/ , setup_output/ , test_images/ . Regenerable — nothing here is an input.

Software/py codes/ — the standalone analysis scripts

19 scripts, one per test series or question, with their own [README.md](#). Run them ****from the repository root****. [v4b_v4c_slope_plot.py](#) and [v4b_vs_v4c_compare.py](#) (the staircase linearity work), [staircase_analysis.py](#) and [staircase_engaged_analysis.py](#), [stress_strain_8_6_19.py](#), [tensile_failure_analysis.py](#), [v5_full_analysis.py](#), [v5_v5b_compare.py](#), [v5abc_compare.py](#), [v6_compare.py](#), [v6_quintet_plots.py](#), [v6_fracture_montage.py](#), the V6a set ([v6a_analyze.py](#), [v6a_plots.py](#), [v6a_epla_offset.py](#), [v6a_offset_window.py](#)), [peak_strain_extraction.py](#), [preload_schedule.py](#) and [generate_report.py](#). Their figures land in [Documentation/Figures/](#).

Software/Matlab/ — where the project started

[UTM.mlapp](#) (the original App Designer application) with [UTM_BAK.mlapp](#) as its backup, [App.m](#) — the exported source, kept for reference during the conversion to Python — and [GUI.png](#) / [MatlabApp.png](#), screenshots of that original interface.

Test Search application/ — a side utility

[find_urls.py](#), [check_urls.py](#) and [master_search.html](#): a small tool for collecting reference URLs and checking that they still resolve. Unrelated to the rig.

Conventions that hold across the whole repository

- **Run scripts from the repository root.** Every data and figure path inside them is written relative to it, not to the script.
- **No number is typed by hand.** Every figure and table in the report and the decks is computed from a raw run by a committed script.
- **`utm\_analysis.py` is the single analyser.** The app and every script import it; nothing re-implements yield, modulus or fracture detection.
- **A missing figure does not fail loudly.** [pic_or_ph\(\)](#) in the deck builders substitutes a placeholder box, so a broken path yields a deck that builds cleanly and is quietly wrong —

check the picture count after moving anything.